

Krishanu Nandan
 ✉ krishanunandan60@gmail.com

Android Development & System Design

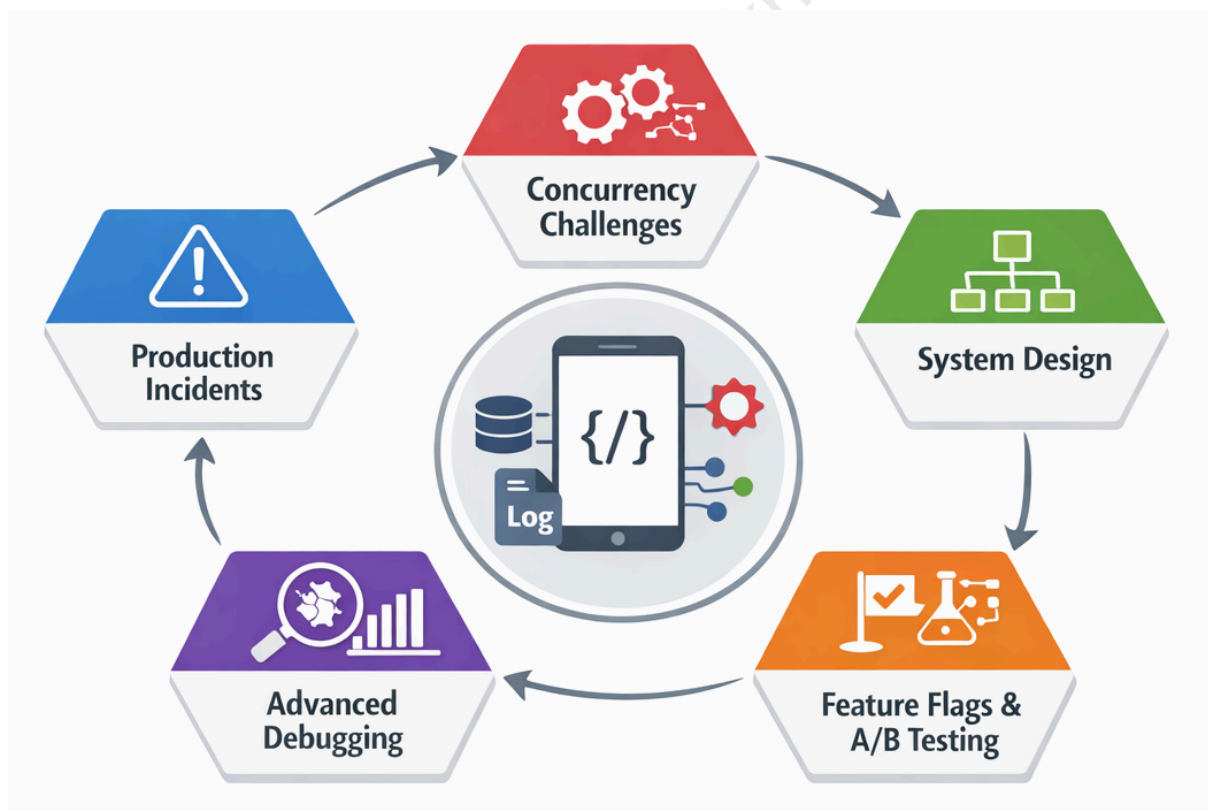
Crafting performant mobile applications with scalable architecture and modern best practices.

Kotlin **Jetpack Compose** **MVVM**

DI **Android SDK** **Java** **Firebase** **Room**

Clean Architecture **SOLID Principles** **Coroutines**

40 Real-World Scenario-Based Interview Questions for Android Engineers



1. Scenario: Memory Leak Causing Gradual App Slowdown

You are working on a large-scale social media app (~20M MAU).

Users are reporting that:

- App becomes slow after 15–20 minutes of usage
- Scrolling starts lagging
- Eventually app gets killed by system (OOM)

Observations from monitoring tools:

- Memory usage continuously increases over time
- GC frequency is very high
- Issue is more prominent on feed screen

Recent changes include:

- New feed redesign using RecyclerView
- Image loading optimizations
- Introduction of a shared singleton analytics manager

How would you investigate and fix this issue end-to-end?

Answer:

First, I would treat this as a **progressive memory leak issue**, not an immediate crash problem, because the degradation happens over time and correlates with user interaction.

1. Confirm Whether It's a Leak or Expected Growth

Before jumping to conclusions, I would validate:

- Is memory growing **linearly without release** → indicates leak
- Or growing and stabilizing → expected caching behavior

Using:

- Android Studio Memory Profiler
- Heap dumps at intervals
- LeakCanary for automatic detection

If objects are retained after screen destruction, it confirms a leak.

Follow [Krishanu Nandan](#)

2. Identify Leak Source Using Heap Analysis

I would capture heap dump and analyze:

- Dominator tree → which objects are retaining memory
- Reference chain → why they are not getting garbage collected

Typical suspects in this scenario:

- RecyclerView Adapter holding reference to Activity/Fragment
 - ViewHolder retaining heavy objects
 - Singleton analytics manager holding Context
 - Image loader caching incorrectly
-

3. Investigate RecyclerView Layer

Since issue is prominent on feed screen:

- Check if adapter is holding strong reference to Context
- Verify if listeners are being cleared in `onViewRecycled()`
- Ensure ViewHolder does not store long-lived references

Also check:

- Are we creating new objects inside `onBindViewHolder()` repeatedly?
-

4. Analyze Singleton / Shared Components

The analytics manager is a strong suspect.

I would verify:

- Is it storing Activity context instead of Application context?
- Is it holding references to views, callbacks, or lifecycle owners?

Fix:

- Replace Activity context with Application context
 - Avoid storing UI references inside singleton
-

5. Image Loading & Caching Layer

- Check if images are being cleared properly
- Ensure lifecycle-aware image loading (e.g., Glide tied to Fragment)

Follow [Krishanu Nandan](#)

- Validate cache size and eviction policy
-

6. GC Pressure Optimization

High GC frequency indicates excessive allocations.

I would:

- Reduce object creation inside scroll path
 - Reuse objects where possible
 - Avoid unnecessary boxing/unboxing
-

7. Fix Strategy Summary

- Remove strong references causing leaks
 - Ensure proper lifecycle cleanup
 - Optimize adapter and ViewHolder usage
 - Fix singleton misuse
 - Tune image caching
-

8. Validation

- Compare heap dumps before and after fix
 - Ensure memory stabilizes over time
 - Monitor GC frequency reduction
 - Run long-session testing (30–60 mins)
-

9. Long-Term Prevention

- Integrate LeakCanary in debug builds
 - Add code review checklist for memory safety
 - Avoid passing Context blindly
 - Introduce architectural boundaries (no UI reference in data layer)
-

Conclusion

This issue is not just a bug but a **systemic lifecycle mismanagement problem**, and solving it requires discipline across UI, architecture, and shared components.

2. Scenario: API Layer Instability (Retries, Failures, Token Expiry)

You are working on a fintech app with millions of daily transactions.

Users report:

- Random API failures
- Some requests succeed on retry
- Occasional logout issues

Observations:

- Increased HTTP 401 and 500 errors
- Multiple duplicate API calls
- Token refresh logic recently modified

Constraints:

- Must ensure **no duplicate financial transactions**
- Backend has rate limits
- Network conditions are unstable (India Tier-2/Tier-3 cities)

How would you design and fix this system?

Answer:

I would approach this as a **network reliability and consistency problem**, especially critical because it involves financial transactions.

1. Categorize Failures

First, I would classify failures into:

- **Client-side issues** (timeouts, retries, duplication)
- **Auth issues** (token expiry, refresh race conditions)
- **Server issues** (500 errors, rate limiting)

This helps avoid mixing multiple root causes.

2. Analyze Token Refresh Flow

Given logout issues and 401 spikes, token handling is a key suspect.

Common issue:

- Multiple requests fail with 401 simultaneously
- Each triggers token refresh → race condition

Fix:

- Implement **single-flight token refresh**

Only one refresh request should execute, others should wait.

3. Prevent Duplicate Requests

Critical for fintech.

I would introduce:

- **Idempotency keys** per request
- Unique transaction IDs generated on client

So even if retry happens:

- Server processes request only once
-

4. Retry Strategy Design

Not all APIs should retry.

- Safe APIs → retry (GET, non-critical POST)
- Financial APIs → controlled retry with idempotency

Use:

- Exponential backoff
 - Network-aware retries
-

5. Network Layer Improvements

- Add OkHttp interceptors:
 - Logging
 - Retry handler
 - Auth handler

- Set proper timeouts:
 - Connection timeout
 - Read timeout
-

6. Rate Limiting Awareness

If backend has limits:

- Avoid aggressive retries
 - Queue requests if needed
 - Use backoff strategy
-

7. Offline Handling

- Queue requests locally (Room DB)
 - Execute when network is available
-

8. Observability

- Add structured logging
 - Track:
 - Retry count
 - Failure rate
 - Token refresh frequency
-

9. Validation

- Simulate poor network conditions
 - Test token expiry edge cases
 - Ensure no duplicate transactions
-

Conclusion

This is not just an API bug — it's a **distributed system reliability issue**, requiring idempotency, synchronization, and controlled retries.

3. Scenario: Offline-First Sync Failure (Message Duplication & Data Loss)

You are building a chat/messaging feature for a large app (~10M DAU), similar to WhatsApp.

Users report:

- Messages sometimes appear duplicated
- Some messages are missing after network recovery
- Message order is inconsistent across devices

Observations:

- App supports offline mode
- Messages are stored locally using Room
- Sync happens via WorkManager
- Backend is eventually consistent

Constraints:

- Messages must **never be lost**
- Duplicate messages are unacceptable
- App must work reliably in poor network conditions

How would you design and fix this system?

Answer:

I would approach this as a **distributed data consistency problem**, not just a mobile bug, because we are dealing with offline-first architecture and eventual consistency.

1. Clarify Data Flow and Failure Points

First, I would map the full lifecycle of a message:

- User sends message → stored locally
- Message marked as **PENDING**
- Sync worker sends to server
- Server responds → message marked as **SENT**

I would identify where duplication or loss can occur:

- Retry logic without idempotency
- Multiple sync workers running concurrently

- Server sending duplicate responses
 - Improper merge logic when syncing back
-

2. Root Cause Analysis

Likely causes:

- No **idempotency key** → same message sent multiple times
 - Sync worker running multiple times → race conditions
 - No proper conflict resolution strategy
 - Local DB not acting as single source of truth
-

3. Fix Strategy — Strong Data Guarantees

a. Introduce Idempotency

Every message must have:

- A unique **client-generated ID (UUID)**

Server must:

- Treat duplicate requests with same ID as same message
-

b. Single Source of Truth (SSOT)

- UI should read only from **local database**
 - Server sync should only update DB, not UI directly
-

c. Sync Queue Design

- Maintain a queue of pending messages
- Ensure only **one worker processes queue at a time**

Use:

- WorkManager with unique work + KEEP policy
-

d. Conflict Resolution

- Use server timestamp as source of truth

- Merge messages carefully to avoid duplication
-

e. Ordering Guarantee

- Maintain logical ordering using:
 - Local timestamp (temporary)
 - Server timestamp (final ordering)
-

4. Retry Strategy

- Use exponential backoff
 - Retry only failed messages
 - Avoid retry storms
-

5. Edge Case Handling

- App killed during sync → WorkManager resumes
 - Network fluctuation → retry safely
 - Partial success → update only successful messages
-

6. Validation

- Simulate:
 - Network drop mid-send
 - Duplicate sends
 - App restarts
 - Ensure:
 - No duplicates
 - No message loss
 - Correct ordering
-

Conclusion

This is fundamentally a **data consistency and synchronization problem**, and the correct solution requires **idempotency, queue control, and strong local-first architecture**.

4. Scenario: Crash Spike Due to Lifecycle Issues (Fragment + Coroutines)

You are working on a modular app with multiple teams contributing.

After a recent release:

- Crash rate increased significantly
- Common crash:
`IllegalStateException: Fragment not attached to a context`

Observations:

- Occurs during navigation or screen rotation
- App uses:
 - Fragments
 - Coroutines
 - ViewBinding

Recent changes:

- Async API calls added inside fragments
- Navigation refactoring

How would you debug and fix this?

Answer:

I would approach this as a **lifecycle misalignment problem between UI components and async operations**.

1. Understand Crash Context

First, I would analyze:

- When does crash occur? → navigation, rotation
- Which thread? → usually main thread
- What operation triggers it? → UI update after async call

This suggests:

- Coroutine completes after Fragment is destroyed
-

2. Root Cause Identification

Typical issue:

- Coroutine launched in Fragment scope
 - Fragment destroyed
 - Coroutine still running
 - On completion → tries to access UI or context
-

3. Fix Strategy — Lifecycle Awareness

a. Use `viewLifecycleOwner` Scope

Instead of:

```
lifecycleScope.launch { ... }
```

Use:

```
viewLifecycleOwner.lifecycleScope.launch { ... }
```

This ensures coroutine is cancelled when view is destroyed.

b. Use `repeatOnLifecycle`

For flows:

```
viewLifecycleOwner.lifecycleScope.launch {  
    repeatOnLifecycle(Lifecycle.State.STARTED) {  
        flow.collect { ... }  
    }  
}
```

c. Avoid Direct Context Usage

Before accessing context:

- Check if fragment is attached
 - Or use `requireContext()` only when safe
-

d. Cancel Jobs Properly

- Store coroutine jobs

- Cancel them in `onDestroyView()` if needed
-

4. Navigation Safety

- Avoid triggering navigation after Fragment is destroyed
 - Use safe navigation patterns
-

5. Architectural Fix

- Move business logic to ViewModel
 - Fragment should only observe state
-

6. Validation

- Test:
 - Rapid navigation
 - Screen rotation
 - Background/foreground
 - Ensure no crashes
-

Conclusion

This issue arises from **mixing asynchronous work with lifecycle-unaware components**, and the fix requires strict lifecycle-scoped execution.

5. Scenario: Slow Build Time in Multi-Module Project

You are working on a large Android codebase:

- 50+ modules
- Multiple teams
- CI build time ~25 minutes
- Local build time ~10–12 minutes

Problems:

- Developers complain about productivity
- Small changes trigger full rebuilds

Follow [Krishanu Nandan](#)

How would you optimize this?

Answer:

I would treat this as a **build system scalability problem**, not just Gradle tuning.

1. Measure Build Bottlenecks

First, I would collect data:

- Use Gradle Build Scan
 - Identify:
 - Longest tasks
 - Non-incremental builds
 - Cache misses
-

2. Identify Root Causes

Common issues:

- Poor module boundaries
 - Too many dependencies between modules
 - Annotation processors (KAPT)
 - Non-incremental tasks
-

3. Modularization Strategy

- Ensure **feature-based modules**
 - Reduce inter-module dependencies
 - Avoid circular dependencies
-

4. Incremental Build Optimization

- Enable incremental compilation
 - Avoid changing shared modules frequently
-

5. Replace KAPT with KSP

- KAPT is slow
 - Migrate to KSP where possible
-

6. Enable Build Cache

- Local + Remote cache
 - Avoid recompilation of unchanged code
-

7. Parallel Execution

- Enable parallel builds
 - Optimize Gradle workers
-

8. Dependency Optimization

- Remove unused dependencies
 - Avoid large libraries
-

9. CI Optimization

- Use remote build cache
 - Run only affected modules
-

Conclusion

Build time issues are usually due to **poor modular boundaries and lack of incremental build optimization**, and solving them requires both architectural and tooling improvements.

6. Scenario: Battery Drain Due to Background Work

You are working on a fitness tracking app.

Users report:

- Significant battery drain
- App appears in top battery usage list

Observations:

- App uses:
 - Location tracking
 - Background sync
 - Periodic API polling

How would you fix this?

Answer:

I would approach this as a **resource efficiency and background execution problem**.

1. Analyze Battery Usage

Use:

- Battery Historian
- Android Profiler

Identify:

- CPU usage
 - Wake locks
 - Network usage
-

2. Identify Problematic Components

Likely causes:

- Frequent location updates
- Continuous background services
- Aggressive polling

3. Fix Strategy

a. Replace Services with WorkManager

- Use WorkManager for deferrable tasks
 - Respect system scheduling
-

b. Optimize Location Updates

- Use balanced accuracy
 - Reduce frequency
-

c. Reduce Polling

- Use push notifications instead of polling
 - Batch network calls
-

d. Respect Doze Mode

- Avoid waking device unnecessarily
-

4. Validation

- Measure battery usage before/after
 - Test long usage scenarios
-

Conclusion

Battery drain issues come from **misuse of background execution**, and the solution is to align with Android's power management system.

7. Scenario: Jetpack Compose Performance Issue (Excessive Recompositions)

You are working on a modern Android app fully built using Jetpack Compose.

Users report:

- UI feels laggy during interactions
- Animations stutter
- CPU usage spikes during scrolling

Observations:

- Recomposition count is very high
- Even small state updates trigger full screen recomposition
- App uses complex UI with nested composables

Recent changes:

- Introduced shared UI state in ViewModel
- Passing large data objects to composables
- Added multiple `collectAsState()` calls

How would you debug and fix this?

Answer:

I would approach this as a **state management and recomposition scope problem**, since Compose performance is tightly coupled with how state is structured and consumed.

1. Measure and Visualize Recompositions

First, I would confirm the issue using:

- Layout Inspector → recomposition count
- Compose tooling (Recomposition highlights)
- CPU profiler

Goal:

- Identify which composables are recomposing frequently
 - Check if recomposition is localized or cascading
-

2. Identify Root Causes

Based on the scenario, likely causes are:

- Passing **unstable or large objects** as parameters
 - Shared state causing **global recomposition**
 - Multiple `collectAsState()` causing redundant updates
 - Missing `remember` or incorrect state scoping
-

3. Fix State Design

a. Hoist and Scope State Properly

- Avoid global state for entire screen
 - Break state into **smaller, independent pieces**
-

b. Use Stable Data Structures

- Ensure models are immutable
 - Avoid passing mutable lists or objects
-

c. Avoid Passing Large Objects

Instead of:

- Passing full UI model

Pass:

- Only required fields
-

4. Optimize State Collection

Instead of multiple:

- `collectAsState()` calls

Use:

- Combine flows in ViewModel
- Expose single UI state

5. Use remember and derivedStateOf

- Cache expensive calculations
 - Avoid recomputation
-

6. Reduce Recomposition Scope

- Break UI into smaller composables
 - Ensure only affected composables recompose
-

7. Advanced Optimization

- Use `key()` for stable identity
 - Avoid lambda recreation inside composables
-

8. Validation

- Compare recomposition counts
 - Measure FPS improvement
 - Track CPU usage
-

Conclusion

Compose performance issues are not UI problems — they are **state architecture problems**, and solving them requires precise control over state flow and recomposition boundaries.

8. Scenario: API Layer Overload (Thundering Herd Problem)

You are working on a news app with millions of users.

At 9 AM daily:

- All users open app
- App triggers API calls for feed

Follow [Krishanu Nandan](https://www.linkedin.com/in/krishanu-nandan/)

Problems:

- Backend gets overloaded
- Many requests fail
- App shows errors or empty data

Observations:

- No caching strategy
- All users hit API simultaneously
- Retry logic increases load

How would you fix this?

Answer:

I would approach this as a **system-level load management problem**, not just an API issue.

1. Identify Root Cause

This is a classic **thundering herd problem**:

- Simultaneous requests from millions of clients
 - No staggering or caching
 - Retry amplifies load
-

2. Introduce Caching Strategy

a. Local Cache (Client-side)

- Store last successful response
 - Show cached data immediately
-

b. Cache Expiry Policy

- Define TTL (e.g., 5–10 minutes)
 - Avoid unnecessary API calls
-

3. Stagger Requests

Follow [Krishanu Nandan](#)

- Introduce random delay before API call
 - Prevent all clients hitting server at once
-

4. Improve Retry Logic

- Use exponential backoff
 - Avoid immediate retries
-

5. Backend Coordination

- Use CDN caching
 - Implement server-side rate limiting
-

6. Smart Fetching

- Fetch only delta updates
 - Avoid full refresh
-

7. Validation

- Simulate peak traffic
 - Monitor API success rate
-

Conclusion

This is not just a mobile issue — it's a **distributed load balancing problem**, requiring both client and server optimizations.

9. Scenario: Deep Link Handling Breaking Navigation

You are working on an e-commerce app.

Users report:

- Deep links sometimes open wrong screen
- App crashes when opened via link
- Back navigation behaves incorrectly

Follow [Krishanu Nandan](https://www.linkedin.com/in/krishanu-android/)

Observations:

- App uses Navigation Component
- Multiple entry points (home, product, offer pages)
- Some deep links contain query parameters

How would you fix this?

Answer:

I would approach this as a **navigation state consistency problem**, especially because deep links bypass normal navigation flow.

1. Understand Deep Link Types

- Cold start deep link
- Warm start deep link
- App already in foreground

Each case behaves differently.

2. Validate Deep Link Parsing

- Ensure URI parsing is correct
 - Validate parameters before using
-

3. Fix Navigation Graph

- Define proper deep link destinations
 - Ensure arguments are correctly mapped
-

4. Handle Back Stack Properly

- Build correct navigation stack manually if needed
 - Avoid duplicate fragments
-

5. Prevent Crashes

Follow [Krishanu Nandan](https://www.linkedin.com/in/krishanu-nandan/)

- Validate data before navigation
 - Handle missing parameters gracefully
-

6. Testing Strategy

- Test:
 - App closed
 - App in background
 - App in foreground
-

Conclusion

Deep linking is not just routing — it's about **reconstructing app state correctly**, and requires careful navigation and validation logic.

10. Scenario: Large List Data Loading Causing OOM

You are building a marketplace app.

Users report:

- App crashes when scrolling large product lists

Observations:

- App loads entire dataset at once
- Images are high resolution
- No pagination implemented

How would you fix this?

Answer:

I would approach this as a **memory management and data loading strategy problem**.

1. Identify Root Cause

- Loading entire dataset → high memory usage

- Large images → memory spikes
 - No lazy loading
-

2. Introduce Pagination

Use:

- Paging 3 library

Benefits:

- Load data incrementally
 - Reduce memory footprint
-

3. Optimize Images

- Resize images before loading
 - Use thumbnails
-

4. RecyclerView Optimization

- Reuse views efficiently
 - Avoid unnecessary object creation
-

5. Cache Strategy

- Use disk + memory cache
 - Avoid reloading images
-

6. Validation

- Monitor memory usage
 - Test with large datasets
-

Conclusion

OOM issues are typically due to **unbounded data loading**, and the solution is controlled, incremental data flow.

<https://www.linkedin.com/in/krishanu-android/>